

Chapter 07

前端框架與現代網站開發

Horazon

互動媒體設計

為什麼需要「框架」？

當網頁越來越複雜 (例如：Facebook, Gmail)，如果只用純 HTML/CSS/JS (Vanilla JS) 開發，會遇到很多問題：

1. **程式碼雜亂**：義大利麵程式碼 (Spaghetti Code)，難以維護。
2. **效能低落**：頻繁操作 DOM 導致網頁卡頓。
3. **重複造輪子**：每個功能都要從頭寫 (例如：登入、表單驗證)。

Framework (框架) 就像是一套標準化的SOP 與 工具包，
幫我們解決這些問題。

現代前端三大巨頭

框架	開發者	特色	學習難度	適用場景
Angular	Google	功能最完整，架構最嚴謹	高 (★★★)	大型企業系統 (銀行、後台)
React	Meta (Facebook)	生態系最大，職缺最多	中 (★★☆)	社群網站、電商、任何應用
Vue	Evan You (個人->社群)	最容易上手，寫法直覺	低 (★☆☆)	中小型專案、個人作品、快速開發

1. Angular

- **誕生**：2010 年 (AngularJS) -> 2016 年 (Angular 2+)。
- **語言**：強制使用 **TypeScript** (JS 的嚴格版)。
- **風格**：All-in-one。
 - 官方把所有東西都準備好了 (路由、表單、HTTP)。
- **優點**：團隊開發規範統一。
- **缺點**：太笨重，學習與設定繁瑣。

2. React

- **誕生**：2013 年。
- **語言**：JavaScript (JSX) / TypeScript。
- **風格**：Component-based (組件化)。
 - 它其實是一個 Library (函式庫)，只負責畫面 (View)。
 - 其他功能 (路由、狀態) 由社群套件補足 (自由度高)。
- **特色**：Virtual DOM (虛擬 DOM)。
- **現況**：目前市佔率第一。

3. Vue.js

- **誕生**：2014 年。
- **語言**：JavaScript / TypeScript。
- **風格**：漸進式框架。
 - 可以像 jQuery 一樣只用在網頁的一部分，也可以做整個 App。
 - 語法結合了 Angular 的模板 (Template) 與 React 的組件概念。
- **特色**：雙向綁定 (Two-way Binding)。
- **現況**：亞洲與華人圈非常流行 (因為文件友善)。

核心概念：組件化 (Component-Based)

這這所有現代框架的共同基礎。

把網頁拆成一個個獨立的 **Component (組件/積木)**。

每個組件包含自己的：

1. **HTML** (結構)
2. **CSS** (樣式)
3. **JS** (邏輯)

好處：可重複使用 (Reusability)。

例如：做一個 `Button` 組件，全站都能用，改一個地方全站都會變。

組件化示意圖

以一個典型的網頁為例：

- **App** (根組件)
 - **Navbar** (導覽列)
 - **Logo**
 - **Menu**
 - **MenuItem**
 - **MenuItem**
 - **MainContent** (主要內容)
 - **ArticleCard** (文章卡片) x 10
 - **Footer** (頁尾)

就像玩 LEGO 樂高積木一樣組合起來！

核心概念：虛擬 DOM (Virtual DOM)

為什麼 React/Vue 速度這麼快？

傳統做法：

JS 直接修改真實 DOM -> 瀏覽器重新繪製整個畫面 -> **慢！**

虛擬 DOM：

1. JS 先在記憶體中建立一個「假的」DOM (Virtual DOM)。
2. 比較新舊 Virtual DOM 的差異 (Diff Algorithm)。
3. **只更新有變動的部分** 到真實 DOM。

比喻：

- **傳統：**為了換一個燈泡，把整棟房子拆掉重蓋。
- **虛擬 DOM：**看著藍圖，只把燈泡換掉。

SPA (Single Page Application)

單頁式應用程式

- 傳統網頁 (Multi-Page) :
 - 點連結 -> 白畫面 -> 重新下載整個頁面。
- SPA (React/Vue) :
 - 就像 Gmail 或 Facebook。
 - 點連結 -> 網址變了，但**頁面沒有重新整理**。
 - 用 JS **局部更新**畫面內容。
 - **優點**：使用者體驗極佳，像 App 一樣流暢。
 - **缺點**：第一次載入較久 (要下載一大包 JS)。

現代開發環境 Ecosystem

要寫現代前端，你需要認識這些工具：

1. Node.js

- 讓電腦可以執行 JavaScript (原本只能在瀏覽器跑)。
- 是用來跑開環境的基石。

2. NPM (Node Package Manager)

- JS 的 App Store。
- 幾百萬個套件任你裝 (例如：輪播圖、日期選擇器)。
- 指令：`npm install react`

資料串接 (API Integration)

前端網頁如何拿到真實資料 (天氣、股價、使用者資料)？

- API (Application Programming Interface)：前後端的橋樑。
- Fetch API：瀏覽器內建的抓資料功能。

```
fetch('https://api.weather.com/taipei')  
  .then(response => response.json())  
  .then(data => {  
    console.log("今日溫度：" + data.temp);  
  });
```

狀態管理 (State Management)

當網頁變複雜，資料 (State) 怎麼管？

- **問題：** A 組件要把資料傳給 B 組件，可能要穿過 10 層 (Props Drilling)。
- **解法：** 把資料放在一個**全域倉庫 (Store)**，誰要用就自己去拿。
- **常見工具：**
 - React: Redux, Zustand
 - Vue: Pinia, Vuex

現代 CSS 趨勢：Tailwind CSS

不想寫 CSS 檔案了？

- Utility-First：直接在 HTML Class 寫樣式。

- 傳統：

```
<div class="card"> (然後去 CSS 寫 .card { padding: 20px; ... })
```

- Tailwind：

```
<div class="p-5 bg-white shadow-lg rounded-lg">
```

- 優點：開發速度極快，不用想 Class 名稱。

JavaScript 的進化：TypeScript

JS 因為太自由，常常寫錯類型 (字串變數字) 導致報錯。
TypeScript (TS) 是微軟開發的「強型別」JS。

```
// JS: 什麼都能裝，容易爆  
let score = 100;  
score = "分"; // 沒問題，但可能導致 Bug  
  
// TS: 嚴格檢查，寫錯 VS Code 會紅字警告  
let score: number = 100;  
score = "分"; // Error: 字串不能裝進數字變數！
```

現在 90% 的大型專案都用 TS 開發。

跨平台開發 (Cross-Platform)

學會了網頁技術 (HTML/CSS/JS)，你不只能寫網頁！

- **手機 App (iOS/Android)：**
 - React Native (React 語法)
 - Ionic (Vue/Angular)
- **電腦軟體 (Windows/Mac)：**
 - Electron (VS Code, Discord 都是用這個寫的！)
 - Tauri (更輕量)

學一套技術，通吃所有平台！

建置工具 (Build Tools)

瀏覽器其實看不懂 `.vue`、`.jsx` 或最新的 JS 語法。

我們需要**編譯器** (Compiler/Bundler) 把它們翻譯成瀏覽器看得懂的 HTML/CSS/JS。

常見工具：

1. Webpack：老牌王者，功能強大但設定超複雜。
2. Vite (法文「快」的意思)：
 - Vue 作者開發的。
 - **速度極快**，現在開發的首選工具。

部署 (Deployment)

寫好網站後，怎麼放上網？

傳統要租主機、設 FTP... 現在不用了！

現代託管平台 (Hosting)：

1. **GitHub Pages**：免費，適合靜態網頁。
2. **Vercel**：Next.js (React框架) 官方平台，速度快，CI/CD 整合好。
3. **Netlify**：老牌靜態託管，功能強大。

流程：

把程式碼推上 **GitHub** -> 連結 **Vercel** -> 自動部署完成！

學習建議 (Learning Path)

如果你想往這條路發展：

1. HTML / CSS / JavaScript 基礎一定要穩！

- 不要一開始就跳框架。
- 如果不懂 JS，寫 React 會很痛苦。

2. 選一個框架深入。

- 推薦 Vue 3 (好上手) 或 React (工作多)。
- 不用全都學 (觀念是通的)。

3. 練習實作。

- 做一個 To-Do List (待辦清單)。
- 做一個天氣查詢網頁 (串接 API)。
- 做一個個人作品集 (Portfolio)。

實作練習：拆解網頁

找一個你喜歡的網站 (例如 YouTube 首頁)。

試著拿紙筆，把它**拆解成組件 (Component)**。

例如：

- **SearchBar** (搜尋框)
- **FilterBadge** (分類標籤)
- **VideoGrid** (影片區塊)
 - **VideoCard** (影片卡片)
 - **Thumbnail** (縮圖)
 - **Avatar** (頭像)
 - **Title** (標題)

這就是前端工程師看網頁的方式！

總結

1. **框架** 解決了大規模開發的難題。
2. React / Vue / Angular 是目前的主流。
3. **組件化 (Component)** 讓程式碼可以像積木一樣重複使用。
4. Virtual DOM 讓網頁變快。
5. SPA 讓網頁像 App 一樣好用。
6. 學習新工具前，**打好 JS 基礎**最重要。

下週，我們將體驗用 AI 來幫我們寫網站！